

Legacy to EKS Migration Plan

Legacy to EKS migration plan

Frame — three unknowns the spec leaves open

§ 1 — Source platform matrix

§ 2 — RPO tightening — strategies to shrink the data-loss window

§ 3 — EKS mounting procedure

§ 4 — Traffic cutover

§ 5 — Rollback strategy

§ 6 — Cross-references

Footnotes — trade-offs, math, edge cases

Legacy to EKS migration plan

Status: Customer-execution playbook (not an internal architecture decision).

Companion to ADR-05 — that ADR establishes Strangler-Fig as the architectural pattern; this doc is the operational playbook that pattern implies.

Audience: the customer's platform / SRE team, or a migration consultant acting on their behalf. § 1 picks the source-platform scenario, § 2 picks the RPO-tightening strategy, § 3 covers the EKS-side procedure, § 4 covers traffic cutover.

Trade-offs and reasoning are pushed to footnotes 1 – 2 at the end.

Frame — three unknowns the spec leaves open

The spec asks: "*migrate live production servers to Kubernetes with minimal impact on daily operations.*" Three things it does NOT pin down — each producing a different right answer.

Unknown	What it determines	Gates which section	Possible answers
WHERE legacy lives	Source platform mechanisms + transfer time	§ 1 source matrix	EC2 / ECS / on-prem / other cloud / different K8s cluster (transfer time differs ~10× across them)
HOW the app wraps LDB	Whether dual-write is feasible at all	§ 2 dual-write strategy	Centralized storage abstraction (1–2 weeks effort) vs LDB calls

Unknown	What it determines	Gates which section	Possible answers
			scattered through business logic (weeks–month refactor)
WHAT "minimal impact" means	RPO budget treatment + acceptable downtime	§ 2 strategy choice + § 3 timing	Zero data loss (forces dual-write or MGN) vs spend RPO budget (cold cutover OK) vs maintenance window (any pattern fine)

Stage 3 conversation should pick one answer in each row before any work starts. This document defaults to the customer treating 6 h RPO as a budget that CAN be spent on planned migration; tighter modes in § 2 cost weeks of refactor in exchange.

§ 1 — Source platform matrix

Pick the row that matches legacy.

Source platform	Sync mechanisms applicable	2 TB cold-migration time	RPO budget consumed	App changes
EC2 / ASG (same AWS account)	EBS Snapshot + Snapshot Copy / AWS MGN	~30–60 min cross-region ³	~40 min of 6 h	None for transfer
EC2 / ASG (different AWS account)	EBS Snapshot cross-account share + Copy	~45–90 min	~45 min	None
On-prem bare-metal	AWS DataSync / Direct Connect rsync / MGN agent	2 TB / 1 Gbps WAN ≈ 5 h ⁴	~5 h	None for transfer; agent install
Other-cloud VM	MGN cross-cloud / DataSync from object storage	4–6 h + cross-cloud egress fees	~5 h + egress cost	None for transfer; agent install
ECS Fargate (EFS-backed)	EFS replication / DataSync EFS → S3 → EKS	~2–4 h	~3 h	None
Existing K8s (different cluster)	Velero CSI restore cross-cluster	~30 min	~30 min	None

Stage 3 question — confirm before any work

Question	Why it matters	Downstream effect
Which row above matches legacy?	Picks the mechanism + transfer time + RPO budget consumed	Determines § 2 strategy options
Is the 6 h RPO ceiling spendable on migration, or strictly reserved for disasters?	If reserved → dual-write or MGN required; if spendable → cold cutover OK	§ 2 strategy + § 3 timing
Is there a customer-accepted maintenance window, or must migration be fully transparent?	Maintenance window relaxes downtime constraints; transparent demands continuous sync	§ 4 cutover pacing

§ 2 — RPO tightening — strategies to shrink the data-loss window

Cold cutover in § 3 has a data-loss window equal to the snapshot-to-mount duration in § 1 (~40 min for EC2, ~5 h for on-prem). Customer may want to tighten this. Four mechanisms — "WAL log-shipping" is NOT one of them ⁵:

Strategy	LDB code	App code	RPO achievable	Real cost	Where it runs
Rsync incremental (file-level)	0	0	30 sec – 2 min (compaction lag)	Compaction storm = full SSTable retransfer	K8s Job in aegis-migration namespace
EBS Snapshot dual-cadence	0	0	5–60 min (snapshot interval)	Storage cost per snapshot; copy time	AWS API; no in-cluster component
AWS MGN block-level continuous	0	0	~10 sec replication lag	Agent install on legacy; AWS MGN per-host fee	AWS MGN service + agent on legacy
App-layer dual-write	0	small–medium ⁶ –	0	Weeks of dual-write SEMANTICS design ⁷ –	App process (legacy AND EKS)

All non-dual-write strategies trigger an fsfreeze per sync event; the kernel → app → ALB → client 5-layer trace is in ⁸ — relevant when freeze duration spikes above the normal sub-second window.

Rsync — runs as K8s Job, not inside the StatefulSet pod

Not inside the main pod (pollutes the image + breaks PSS restricted). Run as a separate Job in aegis-migration namespace:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: leveldb-migration-host-{{ host_id }}
  namespace: aegis-migration
spec:
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - name: rsync-worker
          image: ghcr.io/<org>/alpine-rsync:3.19@sha256:<digest>
          command: ["/bin/sh", "-c"]
          args:
            - |
              rsync -a --delete --partial \
                --rsh="ssh -i /ssh/id_rsa -o StrictHostKeyChecking=no" \
                legacy@$LEGACY_HOST:/var/lib/leveldb/ \
                /mnt/dest/
          env:
            - name: LEGACY_HOST
              valueFrom: { configMapKeyRef: { name: migration-config,
                key: legacy_host }}
          volumeMounts:
            - { name: ssh-key, mountPath: /ssh, readOnly: true }
            - { name: dest-pvc, mountPath: /mnt/dest }
          volumes:
            - name: ssh-key
              secret: { secretName: legacy-ssh-key, defaultMode: 0400 }
            - name: dest-pvc
              persistentVolumeClaim:
                claimName: data-aegis-statefulset-primary-{{ ordinal }}
      ttlSecondsAfterFinished: 3600
```

The destination PVC must exist BEFORE the Job runs — § 3 step 1 handles this. For continuous sync up to cutover, wrap in a CronJob and run the final pass during the cutover window.

§ 3 — EKS mounting procedure

Independent of § 2 strategy. Six steps.

Step 1 — Pre-create destination PVCs (per pod, per volume role)

Path γ multi-PVC per ADR-02 — each pod gets a data and a wal PVC. For N pods, pre-create 2N PVCs before helm install:

```
for i in $(seq 0 $((N-1))); do
  for role in data wal; do
    size=$([ "$role" = "data" ] && echo "1843Gi" || echo "205Gi")
    kubectl apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: $role-aegis-statefulset-primary-$i
  namespace: aegis-app-prod
  labels:
    aegis.io/volume-role: $role
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: ebs-gp3-az-aware
  resources: { requests: { storage: $size } }
EOF
  done
done
```

Shortcut for EC2 source: add `dataSource: { name: <vs>, kind: VolumeSnapshot }` to the PVC spec to provision directly from an EBS Snapshot — skip the sync Job entirely.

Step 2 — Wait for PVCs to bind

```
kubectl wait --for=jsonpath='{.status.phase}'=Bound \
  pvc/data-aegis-statefulset-primary-0 pvc/wal-aegis-statefulset-
  primary-0 \
  -n aegis-app-prod --timeout=5m
```

Step 3 — Run migration Job(s) (skip if PVCs sourced from snapshot)

One Job per pod ordinal, in parallel:

```
for i in $(seq 0 $((N-1))); do
  envsubst < migration-job-template.yaml | sed "s/{{ordinal}}/$i/g" |
  kubectl apply -f -
done
kubectl wait --for=condition=complete \
  job -l app=leveldb-migration -n aegis-migration --timeout=6h
```

Step 4 — Helm install StatefulSet

StatefulSet discovers the pre-created PVCs and binds them. OrderedReady means pod-0 starts first and waits for /ready:

```
helm install aegis-statefulset helm/aegis-statefulset/ \
  -f helm/aegis-statefulset/values-prod.yaml -n aegis-app-prod
```

Step 5 — /ready + smoke test

Production LDB MemTable rebuild at 2 TB takes 5–15 min; startup probe budget is 30 min:

```
kubectl rollout status statefulset/aegis-statefulset-primary \
  -n aegis-app-prod --timeout=30m

for i in $(seq 0 $((N-1))); do
  pod="aegis-statefulset-primary-$i.aegis-statefulset-headless.aegis-
    app-prod"
  ./scripts/chaos/verify-test-data.sh "$pod" "$i" # samples N keys vs
    legacy
done
```

Difference rate must be 0 (cold cutover) or within dual-write lag (continuous sync).

Step 6 — Update placement table (ADR-03)

```
aws dynamodb update-item --table-name placement-table \
  --key "{\"tenant_id\":{\"S\":\"$TENANT\"}}\" \
  --update-expression "SET endpoint = :new" \
  --expression-attribute-values \
    "{\"new\":{\"S\":\"aegis-statefulset-primary-$ORDINAL.aegis-app-
      prod.svc.cluster.local\"}}"
```

API tier placement-table cache TTL (default 5 sec) propagates the change.

§ 4 — Traffic cutover

Two default rules by source. Deviation patterns in ⁹.

Default rule

Source	Default cutover lever	Why
AWS-hosted (EC2 / ECS / existing K8s in AWS)	ALB target group binding (TGB) weighted shift	Same AWS control plane; TGB shift is seconds- reversible; WAF / Shield already in line; zero AWS egress
Non-AWS (on-prem / other cloud / non-AWS K8s)	DNS / edge weighted routing at whichever provider the customer already uses — Route 53, Cloudflare, NS1, Akamai, Google Cloud DNS, Azure Traffic Manager, Fastly, etc.	Zero network plumbing prerequisite; public IPs OK; TTL propagation delay is the only cost; concept is provider-agnostic

Cross-region AWS sources combine both: DNS-layer routing (Route 53 latency routing or equivalent) as outer envelope to pick the region; ALB TGB as inner cutover lever within each region.

Weighted routing alone covers cohort-level cutover (standard Strangler-Fig % shift). Per-tenant granularity needs a programmable layer — typically the ADR-03 placement table at the EKS API tier. Edge scripting (Cloudflare Workers etc.) is an alternative ONLY for the narrow customer profile where it earns its place; see Pattern E in ¹⁰ for the three conditions.

Pattern 4a — ALB TGB weighted shift (AWS source default)

Single ALB, two target groups (legacy + EKS). Phase shifts ¹¹:

Phase	Weight (legacy : EKS)	Soak before next
Phase 0	100 : 0	(baseline)
Phase 1 (canary)	90 : 10	24 h
Phase 2 (cohort)	50 : 50	48 h
Phase 3 (full)	0 : 100	7 days before legacy decommission

L7 connection-drained shift — clients perceive zero connection break.

Pattern 4b — DNS / edge weighted routing (non-AWS source default)

Three-step timing, provider-agnostic — exact API differs (Route 53 change-resource-record-sets, Cloudflare Load Balancing pool weights, NS1 Filter Chain, Akamai Property Manager, etc.) but the timing discipline is the same:

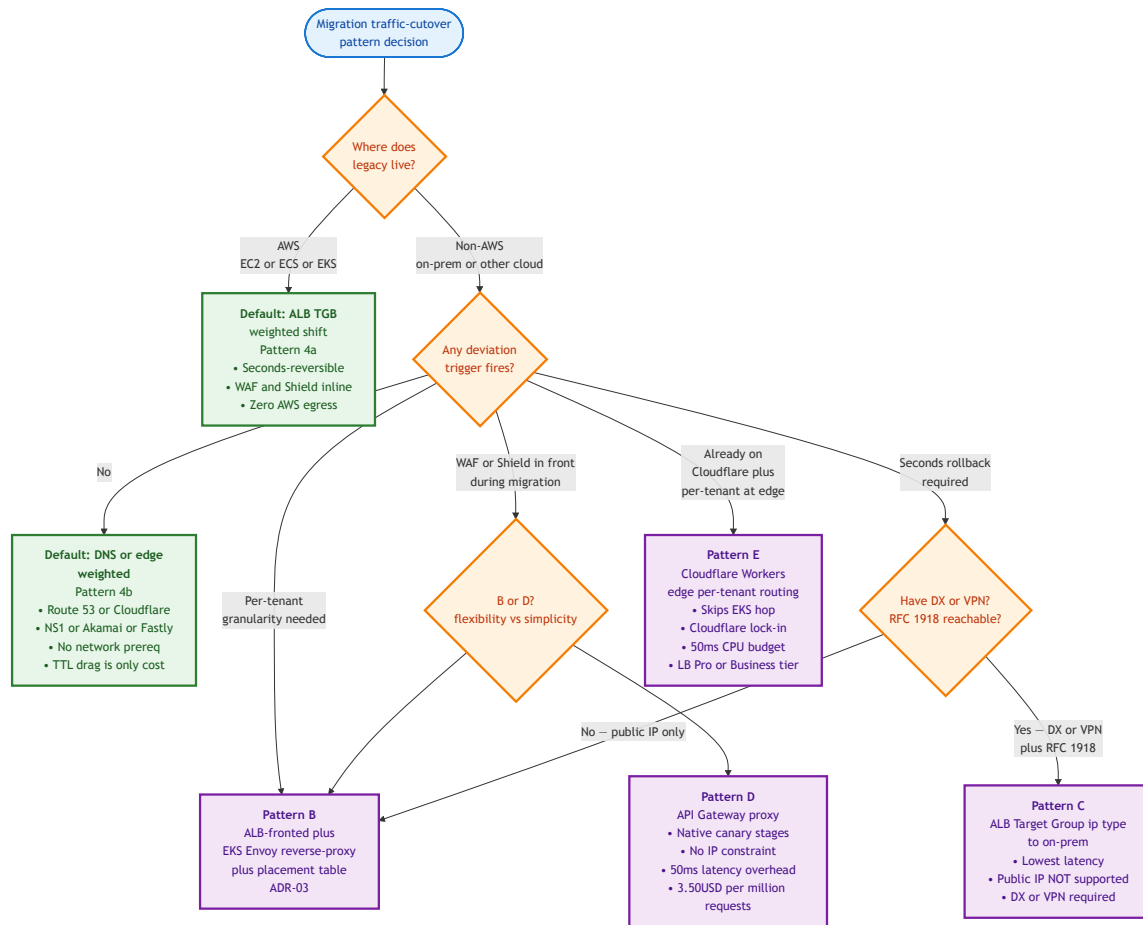
Timing	Action	Why
One week before cutover	Drop record TTL 300 sec → 30 sec	Resolvers refresh once before cutover day; aggressive caches still cap at ~5 min ¹²
Cutover day	Shift weight from legacy origin → new EKS ALB endpoint	Most clients propagate within 30 sec; 5-min "shadow serve" on legacy catches stragglers
One day after stable	Raise TTL back to 300 sec	Reduce DNS query load + resolver pressure

Provider-specific concrete CLI (replace per customer's actual stack):

Provider	API surface
AWS Route 53	aws route53 change-resource-record-sets --hosted-zone-id ... --change-batch ...
Cloudflare	cloudflared / API POST /accounts/.../load_balancers/.../pools with weighted origin pools; or Cloudflare Workers script for per-request routing ¹³
NS1	ns1 record edit ... --weight N

Provider	API surface
Akamai	Property Manager + Fast Purge for TTL flush

Decision flow — visual



When to deviate from default — three triggers

These deviations apply **ONLY** when the customer signals one of these needs; otherwise the default in the first table is correct.

Trigger	Use deviating pattern	Why
Per-tenant canary needed (specific tenants advance / hold independent of cohort)	Pattern B ¹⁴ — ALB-fronted with EKS Envoy reverse proxy + placement-table per-tenant routing (ADR-03)	Route 53 / ALB TGB only do whole-traffic %; only the app-layer placement table knows tenants
AWS WAF / Shield must protect legacy during migration window	Pattern B or D ¹⁵	Pattern A keeps legacy un-fronted until cutover day; B/D move legacy behind AWS edge from day 0
Seconds-level rollback required (regulated industry)	Pattern B, C, or D ¹⁶	DNS TTL drag (30 sec – 5 min) is too slow; API-call rollback is seconds

§ 5 — Rollback strategy

Phase reached	Rollback mechanism	Data-loss cost
Before placement-table flip	Tear down EKS pods + PVCs; legacy still authoritative	Zero
After placement flip, before soak end	Flip placement back to legacy; abandon EKS-side writes	Small window ¹⁷
After soak end	One-way — restore from EKS Velero snapshot if EKS issue surfaces	Last EKS snapshot age ($\leq$ 5 min at default cadence)

§ 6 — Cross-references

Document	Relationship to this playbook
ADR-04	Backup / DR / Three-Layer-DR — the steady-state architecture the customer lands on after migration
ADR-05	Strangler-Fig migration decision — this playbook is the operational mechanics for that ADR
per-tenant-relocation.md	Pattern 1 (shared LDB per pod) vs Pattern 2 (one LDB per tenant) — four-tier framework that gates whether per-tenant cutover is even possible
runbooks/02-aws-bootstrap-and-chaos-demo.md	EKS-side bring-up runbook — what to run before § 3 of this playbook
docs/future/README.md	What happens beyond steady state — T+1y onward, substrate-upgrade triggers at T+3y

Footnotes — trade-offs, math, edge cases

Operational playbook for ADR-05. Architectural decisions live in the ADR; mechanics live here. Trade-off depth lives in the footnotes.

1. EBS Snapshot Copy timing — 2 TB reference table:

Region pair	Wall-clock	Notes
Same region	~10 min	Fastest path
Cross-region (e.g. us-east-1 → eu-central-1)	~30 min	AWS-internal bandwidth, dominated by inter-region link
Cross-region — incremental delta	minutes	Only changed blocks (EBS Snapshot is incremental against prior snapshots from

Region pair	Wall-clock	Notes
same volume)		

Source: [AWS EBS Snapshot Copy docs](#). First migration snapshot is full size; subsequent "final delta" snapshots in a dual-cadence pattern transfer only changed blocks. Actual time varies with snapshot delta size, region pair, AWS-internal throttling.↵

2. Rollback after placement flip — data-loss math by elapsed time:

Time since placement flip	EKS-side accumulated writes	Rollback automatic?	Recovery if customer changes mind
< 5 min	< 1 backup cycle (~1 cadence period)	Yes — flip back, accept loss	Manual; writes not yet in any backup
5 min – 24 h (canary phase)	Captured in EBS Snapshots @ 5-min cadence	Yes — flip back; lose post-snapshot delta only	Velero restore from latest EKS snapshot
24 h – 7 d (cohort/full)	Same as above + cumulative	Yes — but blast radius grows linearly	Velero restore
> 7 d (post-decommission)	Legacy is gone	No — one-way commit point	Only EKS-side disaster recovery applies

Soak window length is the operator's confidence threshold. Longer soak → more EKS-side data accumulates → bigger blast radius on rollback. Typical shape: 24 h canary, 48 h cohort, 7 days full before decommissioning legacy.↵

3. EBS Snapshot Copy timing — 2 TB reference table:

Region pair	Wall-clock	Notes
Same region	~10 min	Fastest path
Cross-region (e.g. us-east-1 → eu-central-1)	~30 min	AWS-internal bandwidth, dominated by inter-region link
Cross-region — incremental delta	minutes	Only changed blocks (EBS Snapshot is incremental against prior snapshots from same volume)

Source: [AWS EBS Snapshot Copy docs](#). First migration snapshot is full size; subsequent "final delta" snapshots in a dual-cadence pattern transfer only changed blocks. Actual time varies with snapshot delta size, region pair, AWS-internal throttling.↵

4. On-prem 2 TB transfer time — bandwidth options:

Network shape	2 TB transfer time	Notes
1 Gbps line rate (theoretical)	4.4 h	$2 \times 10^{12} \times 8 / 10^9 = 16\ 000\ \text{sec}$
1 Gbps real-world (70–80% efficiency)	5–6 h	Saturates 6 h RPO budget; TCP overhead, packet loss recovery
AWS Direct Connect 10 Gbps	~30 min	10× bandwidth; small budget consumption
AWS Snowball Edge	Days wall-clock	Physical ship; zero RPO-budget consumption during transit if combined with continuous sync at the cutover endpoint
Multi-Gbps internet uplink upgrade	Variable	Customer-specific cost depending on local ISP

1 Gbps without DX / Snowball is the worst case — eats the entire 6 h RPO budget on transfer alone, leaving no margin for cutover delta. Direct Connect or Snowball changes the calculus completely.↵

5. **"WAL log-shipping" is not a real option.** A reader might propose "monitor legacy's LevelDB WAL/LOG file and replay into the destination LDB". Three reasons it does not work:

Blocker	Detail
No subscription API	LevelDB's WAL (LOG file) is a per-process crash-recovery artifact; there is no public "subscribe to writes since LSN N" or "replay this WAL on a different instance" API
Internal format, no stability contract	WAL format is version-dependent; parsing WAL bytes directly creates a fragile fork that breaks on every LDB upgrade
Real alternatives both cost more	(a) Patch LDB source to add a hook → maintain a fork forever; (b) wrap LDB at the app layer → that's just dual-write under another name

There is no "free" continuous-sync mechanism for LevelDB; the four strategies in the § 2 table are the real options.↵

6. **Dual-write code-change size depends on app abstraction.** The change size differs by an order of magnitude based on whether the app's LDB access is centralized or scattered:

App structure	Dual-write effort	Failure mode
Centralized storage-access package (clean <code>storage.Write(key, value)</code> interface)	~100–200 LOC in one package + semantics design = 1–2 weeks	All sites covered by changing one package

App structure	Dual-write effort	Failure mode
LDB calls scattered through business logic (<code>leveldb.OpenFile()</code> + <code>db.Put()</code> everywhere)	Weeks to a month of refactor + full call-site audit	A missed call site silently breaks the dual-write invariant — divergent data, hard to debug

This is a Stage 3 question worth asking early because the answer changes the migration timeline by an order of magnitude.↩

7. **Dual-write SEMANTICS — five corner cases the app must decide.** LDB code is unchanged in all dual-write strategies; the harder problem is the SEMANTICS the app's storage layer must handle: (a) **Legacy write succeeds, EKS write fails** — fail the client request (strong consistency, lower availability) vs queue for async retry (higher availability, EKS lag); (b) **Write ordering across both LDBs** — serial dual-write doubles p99 latency; parallel dual-write can diverge order per key on rapid-fire same-key writes, leaving final state inconsistent; (c) **Idempotency on retry** — client request_id + sentinel keys; LDB has no native dedup; (d) **Backfill vs dual-write overlap** — backfill copies pre-flag data, dual-write fires on new data; overlap window needs dedup; (e) **Delete operations** — tombstones must reach both stores or "resurrect" bugs appear. Each one is a real distributed-systems design decision; the LDB code change is the easy part.↩

8. **fsfreeze impact — kernel → app → ALB → client 5-layer trace.** Velero's pre-snapshot hook (or rsync pre-quiesce) triggers a filesystem freeze. Writes are queued, not dropped, at kernel layer; each higher layer adds its own drop conditions.

Layer trace — what happens to a write during freeze:

Layer	Mechanism during freeze	Drop behavior
Kernel syscall	<code>write()</code> / <code>fsync()</code> sleep in waitqueue (D state)	None — pure queue, all writes complete after unfreeze in original order
App handler thread	Worker blocked in <code>db.Put()</code> → blocked in <code>write()</code>	App-internal queue grows; goroutine model accumulates pending I/O
HTTP server	<code>accept()</code> still works (socket syscall, not fs); but workers exhausted at sustained high QPS	503 / connection refused if worker pool maxed
ALB upstream	Pod stays in pool until health check fails (default 5 sec interval × 5 retries = 25 sec)	504 after idle_timeout 60 sec; pod removed after HC failure threshold
Client SDK	Sees high latency, then timeout per its policy	Retry / give up — client-policy dependent

Freeze duration → cross-layer impact at default ALB / Velero config:

Freeze duration	Kernel	App	ALB	Client
< 1 sec (normal)	queue	OK	not noticed	p99 spike to ~1 sec

Freeze duration	Kernel	App	ALB	Client
1–5 sec	queue	some workers blocked	not noticed	p99 spike; some retries fire
5–25 sec	queue	worker pool full → app returns 503	not noticed (HC has not failed yet)	client-side timeouts begin
25–30 sec	queue	503	health check fails → pod removed from TG → traffic dropped	mass timeouts
> 30 sec	queue	503	pod already out of pool	drops

Velero default pre-hook timeout is 30 sec for exactly this reason — it aborts the snapshot before crossing into the "ALB removes pod" zone.

Which § 2 strategy triggers freeze, how often:

§ 2 strategy	Triggers fsfreeze?	Frequency	Mitigation
Rsync incremental	Yes (per rsync run, if quiesce wrapped around it)	Per CronJob tick (e.g. every 15 min during sync window)	Schedule off-peak; coordinate with normal backup cadence
EBS Snapshot dual-cadence	Yes (per snapshot)	Per cadence (5 min operational / 4 h DR)	Same as normal ops — no extra cost during migration
AWS MGN block-level	No	N/A	Block layer below filesystem; MGN's hidden win for freeze-sensitive workloads
App-layer dual-write	No	N/A	Continuous parallel writes; zero quiesce needed

For our default architecture (5-min Velero cadence, Velero pre-hook fsfreeze): every 5 min there's a ~1 sec spike. **0.3% wall-clock at elevated p99**, zero drops under normal disk pressure. The danger zone is only reached when LDB is doing a large MemTable flush or SSTable compaction at the same instant freeze fires — operator should monitor backup duration and alert at 10 sec+ as a degraded signal before reaching the 30-sec Velero abort.↩

9. **Deviation patterns from § 4 default cutover rule.** The default in § 4 (AWS → ALB / non-AWS → Route 53) handles the common cases. Three deviating patterns exist for the three triggers named in § 4:

Pattern B — ALB-fronted with EKS Envoy reverse proxy. Route 53 points at AWS ALB; ALB sends traffic to EKS API tier; EKS Envoy upstream cluster reverse-proxies to legacy. DNS-flip happens early (low-risk one-time event); subsequent traffic shifts happen at ALB TGB or Envoy weight (seconds-reversible). Adds per-

tenant canary capability via the ADR-03 placement table. Cost: one extra hop per request (AWS ↔ legacy network roundtrip, ~5 ms with DX, ~50 ms over public internet) + AWS egress fees per request that hits legacy.

Pattern C — ALB Target Group ip type pointing directly at on-prem / cross-cloud. ALB Target Groups support ip type which can route to RFC 1918 IPs reachable from the VPC. Simpler than Pattern B (no EKS proxy hop) but **requires Direct Connect or Site-to-Site VPN** because public IPs are not supported as IP targets (per [AWS ALB target group docs](#)). Best when migration is cohort-level only (no per-tenant canary need) and customer already has DX/VPN.

Pattern D — API Gateway proxy. API Gateway can backend to any public HTTP URL (no RFC 1918 constraint). Native canary deployment feature on stages. Trade-off: ~50 ms latency overhead vs ALB, pricing is per-request (\$3.50 per million requests + egress) which becomes significant at high QPS. Best when customer already has API Gateway in their stack, or needs the native canary deployment pattern.

Pattern E — Cloudflare Workers edge per-tenant routing (narrow use case). Weighted routing in any DNS / edge provider is enough for cohort-level migration — the standard Strangler-Fig % shift. Programmable edge scripting only adds value when per-tenant granularity is required, and even then our architecture already provides that capability at the EKS API tier via the ADR-03 placement table. Pattern E (Cloudflare Workers) is the right choice only in a narrow customer profile; otherwise it adds complexity without architectural payoff.

When is per-tenant routing actually needed during migration?

Migration intent	Weighted (DNS / ALB) enough?	Programmable needed?
Cohort-level % shift (standard Strangler Fig)	✅ Yes	No
Pin tenant alice to legacy until manual approval	❌ stochastic flapping per-request	Yes
Different cohorts on different schedules	❌	Yes
Identity / region-aware routing	❌ weighted doesn't inspect request content	Yes

Conditions for Pattern E specifically (ALL three must hold):

Condition	Why it matters
Customer is already on Cloudflare Enterprise / Business	Don't introduce a new vendor purely for migration — reuse existing edge
Customer wants per-tenant routing AT THE EDGE, not in AWS API tier	Architectural choice: edge-layer routing saves a hop into AWS but adds vendor lock-in
Customer prefers to avoid building the EKS API tier reverse-proxy	Pattern B is the alternative; both deliver per-tenant routing, just at different layers

If any condition is missing → Pattern E is the wrong choice:

Missing condition	Better alternative
No existing Cloudflare investment	Pattern B (EKS Envoy + placement table) — no new vendor
Only cohort-level needed	Plain weighted DNS / ALB — no programmable layer at all
Going AWS-deep anyway (placement table is being built for steady state)	Pattern B reuses what you build for steady state; Pattern E adds a parallel system

Pattern E trade-offs if picked anyway:

Constraint	Detail
Vendor lock-in	Workers API is Cloudflare-specific; migrating off rewrites the script
CPU budget	Workers default 50 ms; up to 30 sec on paid plan; bounds complex routing logic
Paid tier required	Cloudflare LB needs Pro / Business tier (\$5+/mo to \$200+/mo)
State at edge	No persistent state in Workers itself; need KV / Durable Objects (paid) or external DB for placement data

Bottom line: Pattern E is a real but narrow option. For most migrations on our architecture, **weighted routing (cohort) + ADR-03 placement table (per-tenant via Pattern B's EKS API tier) is the right combination**. Pattern E fits only if the customer has already invested in Cloudflare's edge fabric and explicitly wants the per-tenant logic kept there rather than in AWS.

Decision matrix — when to pick which:

Trigger	Customer's edge	Network prereq	Pick
Per-tenant canary needed	AWS-native or anything else	Any	B (EKS Envoy + placement table — ADR-03 path)
Per-tenant canary needed	Cloudflare	None	E (Workers edge script — skips EKS hop)
WAF / Shield in front during migration	AWS-native	Any	B (most flexible) or D (simplest)
WAF / DDoS in front during migration	Cloudflare	None	E (Cloudflare WAF is integrated)
Seconds rollback, public IP only	Any	No DX/VPN	B (Envoy weight) or D (API GW)
Seconds rollback, RFC 1918 OK	Any	DX/VPN exists	C (simplest, low latency)

Setup time comparison for the deviation patterns:

Pattern	Network prerequisite	Setup time
B (ALB + EKS Envoy)	None (public internet OK) or DX/VPN	Days (Envoy config + EKS bring-up)
C (ALB IP target direct)	DX (weeks) or Site-to-Site VPN (~1 day)	~1 day after network plumbing
D (API Gateway)	None	Hours (API Gateway config)
E (Cloudflare Workers)	None; customer already on Cloudflare	Hours (Worker script + Load Balancer pools)

The default rule (§ 4 first table) is right for ~80% of Mittelstand- scale customers. The five deviation patterns (B / C / D / E plus plain DNS at non-Cloudflare providers) cover the remaining 20% where one of the trigger conditions fires.↩

10. **Deviation patterns from § 4 default cutover rule.** The default in § 4 (AWS → ALB / non-AWS → Route 53) handles the common cases. Three deviating patterns exist for the three triggers named in § 4:

Pattern B — ALB-fronted with EKS Envoy reverse proxy. Route 53 points at AWS ALB; ALB sends traffic to EKS API tier; EKS Envoy upstream cluster reverse-proxies to legacy. DNS-flip happens early (low-risk one-time event); subsequent traffic shifts happen at ALB TGB or Envoy weight (seconds-reversible). Adds per-tenant canary capability via the ADR-03 placement table. Cost: one extra hop per request (AWS ↔ legacy network roundtrip, ~5 ms with DX, ~50 ms over public internet) + AWS egress fees per request that hits legacy.

Pattern C — ALB Target Group ip type pointing directly at on-prem / cross-cloud. ALB Target Groups support ip type which can route to RFC 1918 IPs reachable from the VPC. Simpler than Pattern B (no EKS proxy hop) but **requires Direct Connect or Site-to-Site VPN** because public IPs are not supported as IP targets (per [AWS ALB target group docs](#)). Best when migration is cohort-level only (no per-tenant canary need) and customer already has DX/VPN.

Pattern D — API Gateway proxy. API Gateway can backend to any public HTTP URL (no RFC 1918 constraint). Native canary deployment feature on stages. Trade-off: ~50 ms latency overhead vs ALB, pricing is per-request (\$3.50 per million requests + egress) which becomes significant at high QPS. Best when customer already has API Gateway in their stack, or needs the native canary deployment pattern.

Pattern E — Cloudflare Workers edge per-tenant routing (narrow use case). Weighted routing in any DNS / edge provider is enough for cohort-level migration — the standard Strangler-Fig % shift. Programmable edge scripting only adds value when per-tenant granularity is required, and even then our architecture already provides that capability at the EKS API tier via the ADR-03 placement table. Pattern E (Cloudflare Workers) is the right choice only in a narrow customer profile; otherwise it adds complexity without architectural payoff.

When is per-tenant routing actually needed during migration?

Migration intent	Weighted (DNS / ALB) enough?	Programmable needed?
Cohort-level % shift (standard Strangler Fig)	✅ Yes	No
Pin tenant alice to legacy until manual approval	❌ stochastic flapping per-request	Yes
Different cohorts on different schedules	❌	Yes
Identity / region-aware routing	❌ weighted doesn't inspect request content	Yes

Conditions for Pattern E specifically (ALL three must hold):

Condition	Why it matters
Customer is already on Cloudflare Enterprise / Business	Don't introduce a new vendor purely for migration — reuse existing edge
Customer wants per-tenant routing AT THE EDGE, not in AWS API tier	Architectural choice: edge-layer routing saves a hop into AWS but adds vendor lock-in
Customer prefers to avoid building the EKS API tier reverse-proxy	Pattern B is the alternative; both deliver per-tenant routing, just at different layers

If any condition is missing → Pattern E is the wrong choice:

Missing condition	Better alternative
No existing Cloudflare investment	Pattern B (EKS Envoy + placement table) — no new vendor
Only cohort-level needed	Plain weighted DNS / ALB — no programmable layer at all
Going AWS-deep anyway (placement table is being built for steady state)	Pattern B reuses what you build for steady state; Pattern E adds a parallel system

Pattern E trade-offs if picked anyway:

Constraint	Detail
Vendor lock-in	Workers API is Cloudflare-specific; migrating off rewrites the script
CPU budget	Workers default 50 ms; up to 30 sec on paid plan; bounds complex routing logic
Paid tier required	Cloudflare LB needs Pro / Business tier (\$5+/mo to \$200+/mo)
State at edge	No persistent state in Workers itself; need KV / Durable Objects (paid) or external DB for placement data

Bottom line: Pattern E is a real but narrow option. For most migrations on our architecture, **weighted routing (cohort) + ADR-03 placement table (per-tenant via Pattern B's EKS API tier) is the right combination**. Pattern E fits only if the

customer has already invested in Cloudflare's edge fabric and explicitly wants the per-tenant logic kept there rather than in AWS.

Decision matrix — when to pick which:

Trigger	Customer's edge	Network prereq	Pick
Per-tenant canary needed	AWS-native or anything else	Any	B (EKS Envoy + placement table — ADR-03 path)
Per-tenant canary needed	Cloudflare	None	E (Workers edge script — skips EKS hop)
WAF / Shield in front during migration	AWS-native	Any	B (most flexible) or D (simplest)
WAF / DDoS in front during migration	Cloudflare	None	E (Cloudflare WAF is integrated)
Seconds rollback, public IP only	Any	No DX/VPN	B (Envoy weight) or D (API GW)
Seconds rollback, RFC 1918 OK	Any	DX/VPN exists	C (simplest, low latency)

Setup time comparison for the deviation patterns:

Pattern	Network prerequisite	Setup time
B (ALB + EKS Envoy)	None (public internet OK) or DX/VPN	Days (Envoy config + EKS bring-up)
C (ALB IP target direct)	DX (weeks) or Site-to-Site VPN (~1 day)	~1 day after network plumbing
D (API Gateway)	None	Hours (API Gateway config)
E (Cloudflare Workers)	None; customer already on Cloudflare	Hours (Worker script + Load Balancer pools)

The default rule (§ 4 first table) is right for ~80% of Mittelstand- scale customers. The five deviation patterns (B / C / D / E plus plain DNS at non-Cloudflare providers) cover the remaining 20% where one of the trigger conditions fires. ↩

- ALB TGB phase soak rationale.** Three soak windows balance "detect divergence" vs "migration completion timeline":

Phase	Default soak	What it catches	Higher-confidence tighter	Higher-stakes looser
Canary 10%	24 h	Diurnal traffic patterns — peak hours expose load-dependent bugs that 1 h soak misses	4 h	72 h

Phase	Default soak	What it catches	Higher-confidence tighter	Higher-stakes looser
Cohort 50%	48 h	Weekly-cycle effects at half-rate — divergence affects half the cohort not all	12 h	7 days
Full 100% (pre-decommission)	7 days	Subtle data divergence — e.g. a tenant's monthly batch job surfacing a bug not seen during peak	2 days	30 days

Operator can tighten or loosen all three independently per the customer's risk tolerance. ↩

12. **Route 53 TTL drop rationale (1 week before cutover).** DNS resolvers cache records up to the TTL value. Dropping TTL from 300 sec → 30 sec one week early ensures that by cutover day, every active client has refreshed at least once and is honoring the shorter TTL. 30 sec is the lower bound most public resolvers honor (Cloudflare 1.1.1.1, Google 8.8.8.8). Some corporate / aggressive caching resolvers ignore TTL below their floor (commonly 300 sec) — plan a 5-min "shadow serve" period on legacy on cutover day to catch these clients. After 24 h of stable EKS operation, raise TTL back to 300 sec to reduce DNS query rate and resolver load. ↩
13. **Deviation patterns from § 4 default cutover rule.** The default in § 4 (AWS → ALB / non-AWS → Route 53) handles the common cases. Three deviating patterns exist for the three triggers named in § 4:

Pattern B — ALB-fronted with EKS Envoy reverse proxy. Route 53 points at AWS ALB; ALB sends traffic to EKS API tier; EKS Envoy upstream cluster reverse-proxies to legacy. DNS-flip happens early (low-risk one-time event); subsequent traffic shifts happen at ALB TGB or Envoy weight (seconds-reversible). Adds per-tenant canary capability via the ADR-03 placement table. Cost: one extra hop per request (AWS ↔ legacy network roundtrip, ~5 ms with DX, ~50 ms over public internet) + AWS egress fees per request that hits legacy.

Pattern C — ALB Target Group ip type pointing directly at on-prem / cross-cloud. ALB Target Groups support ip type which can route to RFC 1918 IPs reachable from the VPC. Simpler than Pattern B (no EKS proxy hop) but **requires Direct Connect or Site-to-Site VPN** because public IPs are not supported as IP targets (per [AWS ALB target group docs](#)). Best when migration is cohort-level only (no per-tenant canary need) and customer already has DX/VPN.

Pattern D — API Gateway proxy. API Gateway can backend to any public HTTP URL (no RFC 1918 constraint). Native canary deployment feature on stages. Trade-off: ~50 ms latency overhead vs ALB, pricing is per-request (\$3.50 per million requests + egress) which becomes significant at high QPS. Best when customer already has API Gateway in their stack, or needs the native canary deployment pattern.

Pattern E — Cloudflare Workers edge per-tenant routing (narrow use case). Weighted routing in any DNS / edge provider is enough for cohort-level migration — the standard Strangler-Fig % shift. Programmable edge scripting only adds value when per-tenant granularity is required, and even then our architecture already provides that capability at the EKS API tier via the ADR-03 placement table. Pattern E (Cloudflare Workers) is the right choice only in a narrow customer profile; otherwise it adds complexity without architectural payoff.

When is per-tenant routing actually needed during migration?

Migration intent	Weighted (DNS / ALB) enough?	Programmable needed?
Cohort-level % shift (standard Strangler Fig)	✅ Yes	No
Pin tenant alice to legacy until manual approval	❌ stochastic flapping per-request	Yes
Different cohorts on different schedules	❌	Yes
Identity / region-aware routing	❌ weighted doesn't inspect request content	Yes

Conditions for Pattern E specifically (ALL three must hold):

Condition	Why it matters
Customer is already on Cloudflare Enterprise / Business	Don't introduce a new vendor purely for migration — reuse existing edge
Customer wants per-tenant routing AT THE EDGE, not in AWS API tier	Architectural choice: edge-layer routing saves a hop into AWS but adds vendor lock-in
Customer prefers to avoid building the EKS API tier reverse-proxy	Pattern B is the alternative; both deliver per-tenant routing, just at different layers

If any condition is missing → Pattern E is the wrong choice:

Missing condition	Better alternative
No existing Cloudflare investment	Pattern B (EKS Envoy + placement table) — no new vendor
Only cohort-level needed	Plain weighted DNS / ALB — no programmable layer at all
Going AWS-deep anyway (placement table is being built for steady state)	Pattern B reuses what you build for steady state; Pattern E adds a parallel system

Pattern E trade-offs if picked anyway:

Constraint	Detail
Vendor lock-in	Workers API is Cloudflare-specific; migrating off rewrites the script

Constraint	Detail
CPU budget	Workers default 50 ms; up to 30 sec on paid plan; bounds complex routing logic
Paid tier required	Cloudflare LB needs Pro / Business tier (\$5+/mo to \$200+/mo)
State at edge	No persistent state in Workers itself; need KV / Durable Objects (paid) or external DB for placement data

Bottom line: Pattern E is a real but narrow option. For most migrations on our architecture, **weighted routing (cohort) + ADR-03 placement table (per-tenant via Pattern B's EKS API tier) is the right combination**. Pattern E fits only if the customer has already invested in Cloudflare's edge fabric and explicitly wants the per-tenant logic kept there rather than in AWS.

Decision matrix — when to pick which:

Trigger	Customer's edge	Network prereq	Pick
Per-tenant canary needed	AWS-native or anything else	Any	B (EKS Envoy + placement table — ADR-03 path)
Per-tenant canary needed	Cloudflare	None	E (Workers edge script — skips EKS hop)
WAF / Shield in front during migration	AWS-native	Any	B (most flexible) or D (simplest)
WAF / DDoS in front during migration	Cloudflare	None	E (Cloudflare WAF is integrated)
Seconds rollback, public IP only	Any	No DX/VPN	B (Envoy weight) or D (API GW)
Seconds rollback, RFC 1918 OK	Any	DX/VPN exists	C (simplest, low latency)

Setup time comparison for the deviation patterns:

Pattern	Network prerequisite	Setup time
B (ALB + EKS Envoy)	None (public internet OK) or DX/VPN	Days (Envoy config + EKS bring-up)
C (ALB IP target direct)	DX (weeks) or Site-to-Site VPN (~1 day)	~1 day after network plumbing
D (API Gateway)	None	Hours (API Gateway config)
E (Cloudflare Workers)	None; customer already on Cloudflare	Hours (Worker script + Load Balancer pools)

The default rule (§ 4 first table) is right for ~80% of Mittelstand- scale customers. The five deviation patterns (B / C / D / E plus plain DNS at non-Cloudflare providers) cover the remaining 20% where one of the trigger conditions fires. ↩

14. **Deviation patterns from § 4 default cutover rule.** The default in § 4 (AWS → ALB / non-AWS → Route 53) handles the common cases. Three deviating patterns exist for the three triggers named in § 4:

Pattern B — ALB-fronted with EKS Envoy reverse proxy. Route 53 points at AWS ALB; ALB sends traffic to EKS API tier; EKS Envoy upstream cluster reverse-proxies to legacy. DNS-flip happens early (low-risk one-time event); subsequent traffic shifts happen at ALB TGB or Envoy weight (seconds-reversible). Adds per-tenant canary capability via the ADR-03 placement table. Cost: one extra hop per request (AWS ↔ legacy network roundtrip, ~5 ms with DX, ~50 ms over public internet) + AWS egress fees per request that hits legacy.

Pattern C — ALB Target Group ip type pointing directly at on-prem / cross-cloud. ALB Target Groups support ip type which can route to RFC 1918 IPs reachable from the VPC. Simpler than Pattern B (no EKS proxy hop) but **requires Direct Connect or Site-to-Site VPN** because public IPs are not supported as IP targets (per [AWS ALB target group docs](#)). Best when migration is cohort-level only (no per-tenant canary need) and customer already has DX/VPN.

Pattern D — API Gateway proxy. API Gateway can backend to any public HTTP URL (no RFC 1918 constraint). Native canary deployment feature on stages. Trade-off: ~50 ms latency overhead vs ALB, pricing is per-request (\$3.50 per million requests + egress) which becomes significant at high QPS. Best when customer already has API Gateway in their stack, or needs the native canary deployment pattern.

Pattern E — Cloudflare Workers edge per-tenant routing (narrow use case). Weighted routing in any DNS / edge provider is enough for cohort-level migration — the standard Strangler-Fig % shift. Programmable edge scripting only adds value when per-tenant granularity is required, and even then our architecture already provides that capability at the EKS API tier via the ADR-03 placement table. Pattern E (Cloudflare Workers) is the right choice only in a narrow customer profile; otherwise it adds complexity without architectural payoff.

When is per-tenant routing actually needed during migration?

Migration intent	Weighted (DNS / ALB) enough?	Programmable needed?
Cohort-level % shift (standard Strangler Fig)	✅ Yes	No
Pin tenant alice to legacy until manual approval	❌ stochastic flapping per-request	Yes
Different cohorts on different schedules	❌	Yes
Identity / region-aware routing	❌ weighted doesn't inspect request content	Yes

Conditions for Pattern E specifically (ALL three must hold):

Condition	Why it matters
Customer is already on Cloudflare Enterprise / Business	Don't introduce a new vendor purely for migration — reuse existing edge
Customer wants per-tenant routing AT THE EDGE, not in AWS API tier	Architectural choice: edge-layer routing saves a hop into AWS but adds vendor lock-in
Customer prefers to avoid building the EKS API tier reverse-proxy	Pattern B is the alternative; both deliver per-tenant routing, just at different layers

If any condition is missing → Pattern E is the wrong choice:

Missing condition	Better alternative
No existing Cloudflare investment	Pattern B (EKS Envoy + placement table) — no new vendor
Only cohort-level needed	Plain weighted DNS / ALB — no programmable layer at all
Going AWS-deep anyway (placement table is being built for steady state)	Pattern B reuses what you build for steady state; Pattern E adds a parallel system

Pattern E trade-offs if picked anyway:

Constraint	Detail
Vendor lock-in	Workers API is Cloudflare-specific; migrating off rewrites the script
CPU budget	Workers default 50 ms; up to 30 sec on paid plan; bounds complex routing logic
Paid tier required	Cloudflare LB needs Pro / Business tier (\$5+/mo to \$200+/mo)
State at edge	No persistent state in Workers itself; need KV / Durable Objects (paid) or external DB for placement data

Bottom line: Pattern E is a real but narrow option. For most migrations on our architecture, **weighted routing (cohort) + ADR-03 placement table (per-tenant via Pattern B's EKS API tier) is the right combination**. Pattern E fits only if the customer has already invested in Cloudflare's edge fabric and explicitly wants the per-tenant logic kept there rather than in AWS.

Decision matrix — when to pick which:

Trigger	Customer's edge	Network prereq	Pick
Per-tenant canary needed	AWS-native or anything else	Any	B (EKS Envoy + placement table — ADR-03 path)
Per-tenant canary needed	Cloudflare	None	E (Workers edge script — skips EKS hop)
WAF / Shield in front during	AWS-native	Any	B (most flexible) or D (simplest)

Trigger	Customer's edge	Network prereq	Pick
migration			
WAF / DDoS in front during migration	Cloudflare	None	E (Cloudflare WAF is integrated)
Seconds rollback, public IP only	Any	No DX/VPN	B (Envoy weight) or D (API GW)
Seconds rollback, RFC 1918 OK	Any	DX/VPN exists	C (simplest, low latency)

Setup time comparison for the deviation patterns:

Pattern	Network prerequisite	Setup time
B (ALB + EKS Envoy)	None (public internet OK) or DX/VPN	Days (Envoy config + EKS bring-up)
C (ALB IP target direct)	DX (weeks) or Site-to-Site VPN (~1 day)	~1 day after network plumbing
D (API Gateway)	None	Hours (API Gateway config)
E (Cloudflare Workers)	None; customer already on Cloudflare	Hours (Worker script + Load Balancer pools)

The default rule (§ 4 first table) is right for ~80% of Mittelstand- scale customers. The five deviation patterns (B / C / D / E plus plain DNS at non-Cloudflare providers) cover the remaining 20% where one of the trigger conditions fires.↩

- Deviation patterns from § 4 default cutover rule.** The default in § 4 (AWS → ALB / non-AWS → Route 53) handles the common cases. Three deviating patterns exist for the three triggers named in § 4:

Pattern B — ALB-fronted with EKS Envoy reverse proxy. Route 53 points at AWS ALB; ALB sends traffic to EKS API tier; EKS Envoy upstream cluster reverse-proxies to legacy. DNS-flip happens early (low-risk one-time event); subsequent traffic shifts happen at ALB TGB or Envoy weight (seconds-reversible). Adds per-tenant canary capability via the ADR-03 placement table. Cost: one extra hop per request (AWS ↔ legacy network roundtrip, ~5 ms with DX, ~50 ms over public internet) + AWS egress fees per request that hits legacy.

Pattern C — ALB Target Group ip type pointing directly at on-prem / cross-cloud. ALB Target Groups support ip type which can route to RFC 1918 IPs reachable from the VPC. Simpler than Pattern B (no EKS proxy hop) but **requires Direct Connect or Site-to-Site VPN** because public IPs are not supported as IP targets (per [AWS ALB target group docs](#)). Best when migration is cohort-level only (no per-tenant canary need) and customer already has DX/VPN.

Pattern D — API Gateway proxy. API Gateway can backend to any public HTTP URL (no RFC 1918 constraint). Native canary deployment feature on stages. Trade-off: ~50 ms latency overhead vs ALB, pricing is per-request (\$3.50 per million requests + egress) which becomes significant at high QPS. Best when customer already has API Gateway in their stack, or needs the native canary deployment pattern.

Pattern E — Cloudflare Workers edge per-tenant routing (narrow use case). Weighted routing in any DNS / edge provider is enough for cohort-level migration — the standard Strangler-Fig % shift. Programmable edge scripting only adds value when per-tenant granularity is required, and even then our architecture already provides that capability at the EKS API tier via the ADR-03 placement table. Pattern E (Cloudflare Workers) is the right choice only in a narrow customer profile; otherwise it adds complexity without architectural payoff.

When is per-tenant routing actually needed during migration?

Migration intent	Weighted (DNS / ALB) enough?	Programmable needed?
Cohort-level % shift (standard Strangler Fig)	✓ Yes	No
Pin tenant alice to legacy until manual approval	✗ stochastic flapping per-request	Yes
Different cohorts on different schedules	✗	Yes
Identity / region-aware routing	✗ weighted doesn't inspect request content	Yes

Conditions for Pattern E specifically (ALL three must hold):

Condition	Why it matters
Customer is already on Cloudflare Enterprise / Business	Don't introduce a new vendor purely for migration — reuse existing edge
Customer wants per-tenant routing AT THE EDGE, not in AWS API tier	Architectural choice: edge-layer routing saves a hop into AWS but adds vendor lock-in
Customer prefers to avoid building the EKS API tier reverse-proxy	Pattern B is the alternative; both deliver per-tenant routing, just at different layers

If any condition is missing → Pattern E is the wrong choice:

Missing condition	Better alternative
No existing Cloudflare investment	Pattern B (EKS Envoy + placement table) — no new vendor
Only cohort-level needed	Plain weighted DNS / ALB — no programmable layer at all
Going AWS-deep anyway (placement table is being built for steady state)	Pattern B reuses what you build for steady state; Pattern E adds a parallel system

Pattern E trade-offs if picked anyway:

Constraint	Detail
Vendor lock-in	Workers API is Cloudflare-specific; migrating off rewrites the script

Constraint	Detail
CPU budget	Workers default 50 ms; up to 30 sec on paid plan; bounds complex routing logic
Paid tier required	Cloudflare LB needs Pro / Business tier (\$5+/mo to \$200+/mo)
State at edge	No persistent state in Workers itself; need KV / Durable Objects (paid) or external DB for placement data

Bottom line: Pattern E is a real but narrow option. For most migrations on our architecture, **weighted routing (cohort) + ADR-03 placement table (per-tenant via Pattern B's EKS API tier) is the right combination**. Pattern E fits only if the customer has already invested in Cloudflare's edge fabric and explicitly wants the per-tenant logic kept there rather than in AWS.

Decision matrix — when to pick which:

Trigger	Customer's edge	Network prereq	Pick
Per-tenant canary needed	AWS-native or anything else	Any	B (EKS Envoy + placement table — ADR-03 path)
Per-tenant canary needed	Cloudflare	None	E (Workers edge script — skips EKS hop)
WAF / Shield in front during migration	AWS-native	Any	B (most flexible) or D (simplest)
WAF / DDoS in front during migration	Cloudflare	None	E (Cloudflare WAF is integrated)
Seconds rollback, public IP only	Any	No DX/VPN	B (Envoy weight) or D (API GW)
Seconds rollback, RFC 1918 OK	Any	DX/VPN exists	C (simplest, low latency)

Setup time comparison for the deviation patterns:

Pattern	Network prerequisite	Setup time
B (ALB + EKS Envoy)	None (public internet OK) or DX/VPN	Days (Envoy config + EKS bring-up)
C (ALB IP target direct)	DX (weeks) or Site-to-Site VPN (~1 day)	~1 day after network plumbing
D (API Gateway)	None	Hours (API Gateway config)
E (Cloudflare Workers)	None; customer already on Cloudflare	Hours (Worker script + Load Balancer pools)

The default rule (§ 4 first table) is right for ~80% of Mittelstand- scale customers. The five deviation patterns (B / C / D / E plus plain DNS at non-Cloudflare providers) cover the remaining 20% where one of the trigger conditions fires. ↩

16. **Deviation patterns from § 4 default cutover rule.** The default in § 4 (AWS → ALB / non-AWS → Route 53) handles the common cases. Three deviating patterns exist for the three triggers named in § 4:

Pattern B — ALB-fronted with EKS Envoy reverse proxy. Route 53 points at AWS ALB; ALB sends traffic to EKS API tier; EKS Envoy upstream cluster reverse-proxies to legacy. DNS-flip happens early (low-risk one-time event); subsequent traffic shifts happen at ALB TGB or Envoy weight (seconds-reversible). Adds per-tenant canary capability via the ADR-03 placement table. Cost: one extra hop per request (AWS ↔ legacy network roundtrip, ~5 ms with DX, ~50 ms over public internet) + AWS egress fees per request that hits legacy.

Pattern C — ALB Target Group ip type pointing directly at on-prem / cross-cloud. ALB Target Groups support ip type which can route to RFC 1918 IPs reachable from the VPC. Simpler than Pattern B (no EKS proxy hop) but **requires Direct Connect or Site-to-Site VPN** because public IPs are not supported as IP targets (per [AWS ALB target group docs](#)). Best when migration is cohort-level only (no per-tenant canary need) and customer already has DX/VPN.

Pattern D — API Gateway proxy. API Gateway can backend to any public HTTP URL (no RFC 1918 constraint). Native canary deployment feature on stages. Trade-off: ~50 ms latency overhead vs ALB, pricing is per-request (\$3.50 per million requests + egress) which becomes significant at high QPS. Best when customer already has API Gateway in their stack, or needs the native canary deployment pattern.

Pattern E — Cloudflare Workers edge per-tenant routing (narrow use case). Weighted routing in any DNS / edge provider is enough for cohort-level migration — the standard Strangler-Fig % shift. Programmable edge scripting only adds value when per-tenant granularity is required, and even then our architecture already provides that capability at the EKS API tier via the ADR-03 placement table. Pattern E (Cloudflare Workers) is the right choice only in a narrow customer profile; otherwise it adds complexity without architectural payoff.

When is per-tenant routing actually needed during migration?

Migration intent	Weighted (DNS / ALB) enough?	Programmable needed?
Cohort-level % shift (standard Strangler Fig)	✅ Yes	No
Pin tenant alice to legacy until manual approval	❌ stochastic flapping per-request	Yes
Different cohorts on different schedules	❌	Yes
Identity / region-aware routing	❌ weighted doesn't inspect request content	Yes

Conditions for Pattern E specifically (ALL three must hold):

Condition	Why it matters
Customer is already on Cloudflare Enterprise / Business	Don't introduce a new vendor purely for migration — reuse existing edge
Customer wants per-tenant routing AT THE EDGE, not in AWS API tier	Architectural choice: edge-layer routing saves a hop into AWS but adds vendor lock-in
Customer prefers to avoid building the EKS API tier reverse-proxy	Pattern B is the alternative; both deliver per-tenant routing, just at different layers

If any condition is missing → Pattern E is the wrong choice:

Missing condition	Better alternative
No existing Cloudflare investment	Pattern B (EKS Envoy + placement table) — no new vendor
Only cohort-level needed	Plain weighted DNS / ALB — no programmable layer at all
Going AWS-deep anyway (placement table is being built for steady state)	Pattern B reuses what you build for steady state; Pattern E adds a parallel system

Pattern E trade-offs if picked anyway:

Constraint	Detail
Vendor lock-in	Workers API is Cloudflare-specific; migrating off rewrites the script
CPU budget	Workers default 50 ms; up to 30 sec on paid plan; bounds complex routing logic
Paid tier required	Cloudflare LB needs Pro / Business tier (\$5+/mo to \$200+/mo)
State at edge	No persistent state in Workers itself; need KV / Durable Objects (paid) or external DB for placement data

Bottom line: Pattern E is a real but narrow option. For most migrations on our architecture, **weighted routing (cohort) + ADR-03 placement table (per-tenant via Pattern B's EKS API tier) is the right combination**. Pattern E fits only if the customer has already invested in Cloudflare's edge fabric and explicitly wants the per-tenant logic kept there rather than in AWS.

Decision matrix — when to pick which:

Trigger	Customer's edge	Network prereq	Pick
Per-tenant canary needed	AWS-native or anything else	Any	B (EKS Envoy + placement table — ADR-03 path)
Per-tenant canary needed	Cloudflare	None	E (Workers edge script — skips EKS hop)
WAF / Shield in front during	AWS-native	Any	B (most flexible) or D (simplest)

Trigger	Customer's edge	Network prereq	Pick
migration			
WAF / DDoS in front during migration	Cloudflare	None	E (Cloudflare WAF is integrated)
Seconds rollback, public IP only	Any	No DX/VPN	B (Envoy weight) or D (API GW)
Seconds rollback, RFC 1918 OK	Any	DX/VPN exists	C (simplest, low latency)

Setup time comparison for the deviation patterns:

Pattern	Network prerequisite	Setup time
B (ALB + EKS Envoy)	None (public internet OK) or DX/VPN	Days (Envoy config + EKS bring-up)
C (ALB IP target direct)	DX (weeks) or Site-to-Site VPN (~1 day)	~1 day after network plumbing
D (API Gateway)	None	Hours (API Gateway config)
E (Cloudflare Workers)	None; customer already on Cloudflare	Hours (Worker script + Load Balancer pools)

The default rule (§ 4 first table) is right for ~80% of Mittelstand- scale customers. The five deviation patterns (B / C / D / E plus plain DNS at non-Cloudflare providers) cover the remaining 20% where one of the trigger conditions fires.↩

17. Rollback after placement flip — data-loss math by elapsed time:

Time since placement flip	EKS-side accumulated writes	Rollback automatic?	Recovery if customer changes mind
< 5 min	< 1 backup cycle (~1 cadence period)	Yes — flip back, accept loss	Manual; writes not yet in any backup
5 min – 24 h (canary phase)	Captured in EBS Snapshots @ 5-min cadence	Yes — flip back; lose post-snapshot delta only	Velero restore from latest EKS snapshot
24 h – 7 d (cohort/full)	Same as above + cumulative	Yes — but blast radius grows linearly	Velero restore
> 7 d (post-decommission)	Legacy is gone	No — one-way commit point	Only EKS-side disaster recovery applies

Soak window length is the operator's confidence threshold. Longer soak → more EKS-side data accumulates → bigger blast radius on rollback. Typical shape: 24 h canary, 48 h cohort, 7 days full before decommissioning legacy.↩

